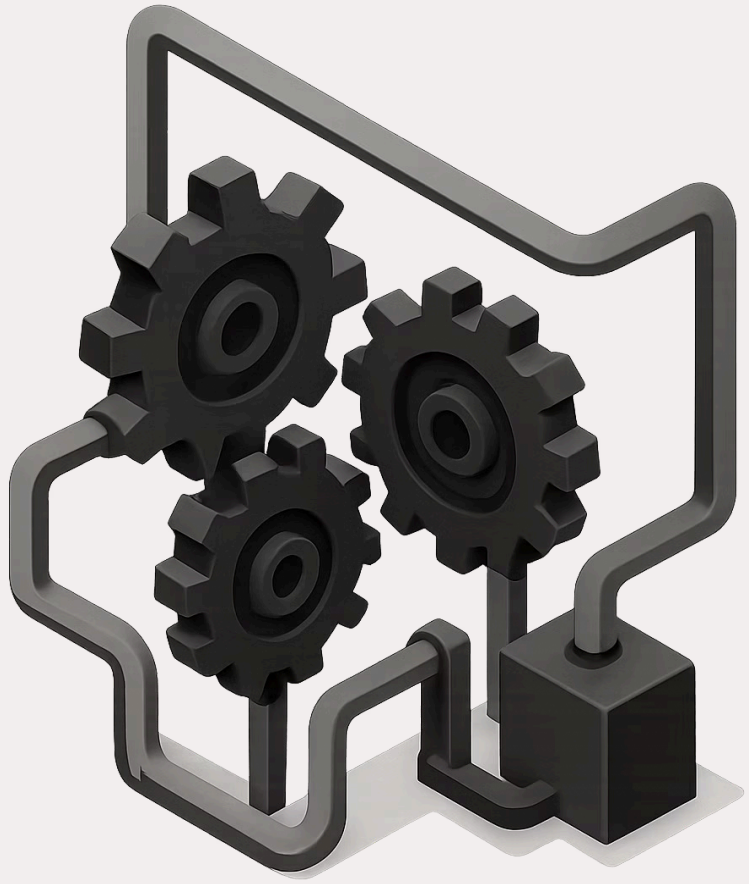
Airflow Core Internals: A Deep Dive

A technical exploration of the scheduler, DAG parsing, task lifecycle, executor models, metadata schema, and configuration — everything you need to truly understand what happens under the hood.



CHAPTER 1

The Orchestration Engine

Scheduler Architecture

The scheduler is the heartbeat of every Airflow deployment — continuously monitoring, evaluating, and triggering work across your data pipelines.

The Heartbeat of Airflow: The Scheduler

The Airflow scheduler is a persistent, always-running service that acts as the central nervous system of any Airflow deployment. Launched via the `airflow scheduler` command, it continuously monitors all registered DAGs and their associated task instances, triggering execution the moment all upstream dependencies have been satisfied.

Persistent Service

Runs continuously in the background, restarting automatically if interrupted. Never stops scanning for schedulable work.

Dependency Awareness

Collects DAG parsing results and evaluates inter-task dependencies before triggering any task instance for execution.

Scheduling Cadence

Performs scheduling evaluations approximately once per minute by default, ensuring timely task dispatch across all active DAGs.

High Throughput Design

The scheduler is purpose-built for speed — minimizing latency between a task becoming eligible and its actual execution. At each scheduling loop iteration, the scheduler checks all active pools for available slots and enqueues up to that many tasks simultaneously.

When demand exceeds capacity, **task priority weights** determine queue order — ensuring critical pipelines are never starved by lower-priority workloads.

1 Check Pool Slots

Evaluate available concurrency slots across all configured pools before scheduling.

2 Enqueue Eligible Tasks

Dispatch up to the pool slot limit per scheduling iteration for maximum throughput.

3 Apply Priority Weights

When tasks exceed slots, priority weights govern which tasks get scheduled first.

High Availability (HA) Scheduler

Airflow supports running multiple scheduler instances simultaneously, enabling true high availability without complex external coordination. The metadata database serves as the single source of truth, with each scheduler using **serialized DAG representations** to make consistent, conflict-free scheduling decisions.

1

Create DagRuns

Identify DAGs that need a new run based on schedule interval and create corresponding DagRun records.

2

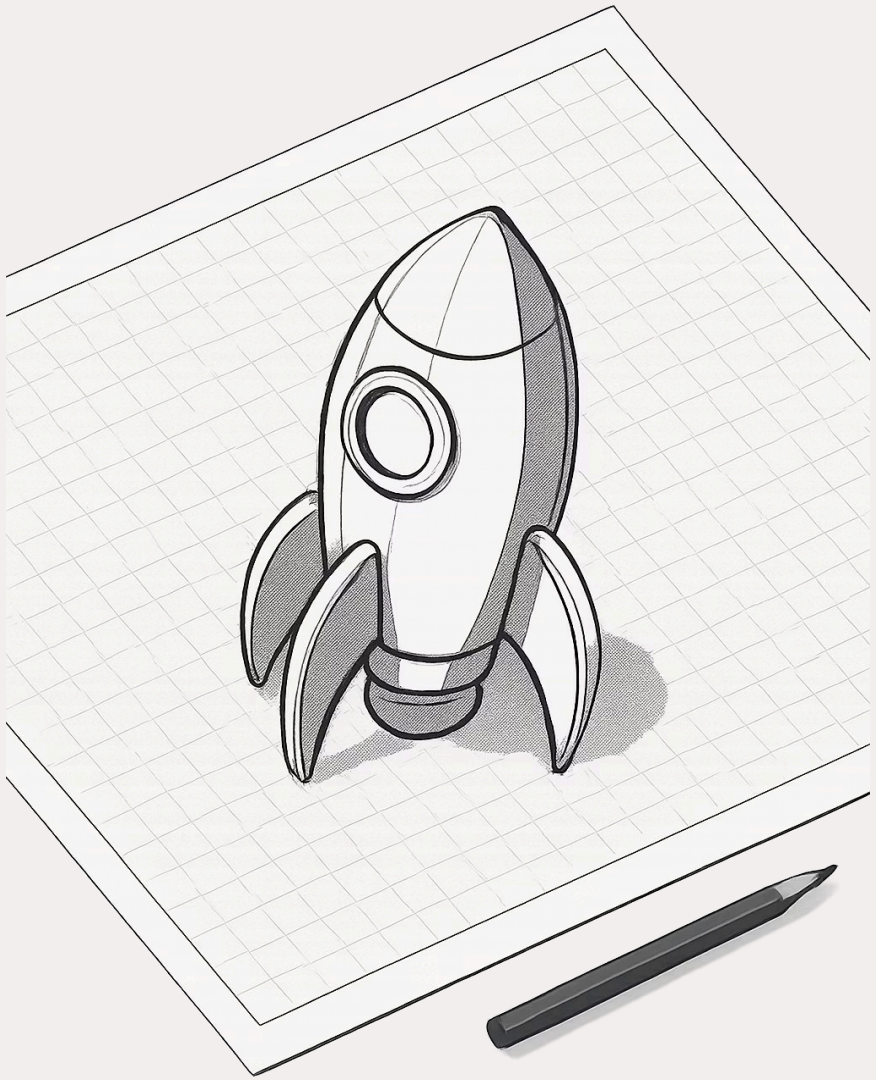
Examine DagRuns

Inspect active DagRuns and evaluate which TaskInstances are now eligible based on dependency resolution.

3

Enqueue Tasks

Place schedulable TaskInstances into the executor queue, respecting pool limits and concurrency settings.



CHAPTER 2

Bringing Workflows to Life

DAG Parsing Mechanism

DAGs are Python files that define the structure, dependencies, and schedule of your workflows. Understanding how Airflow reads and interprets them is key to writing reliable pipelines.

The Blueprint of Workflows: DAG Parsing

Airflow continuously reads a designated **DAGs folder** — scanning Python files to discover and register workflow definitions. Each file is parsed by a DagFileProcessor, which imports the module, extracts DAG objects, and serializes them into the metadata database for the scheduler to consume.

DAGs define the **dependency graph** between tasks, while individual tasks specify the actual work: fetching data, running SQL, executing shell commands, calling APIs, and more. The separation of orchestration logic from execution logic is what makes Airflow so powerful.

📁 DAGs Folder

Watched directory containing Python workflow definition files.

🔗 Dependencies

DAGs encode task ordering via upstream/downstream relationships.

⚙️ Task Operators

Tasks define the actual work: SQL, Python, Bash, HTTP, and more.

Dynamic DAG Generation

One of Airflow's most powerful features is the ability to **generate DAGs programmatically** at parse time. Because DAG files are plain Python, you can loop over configuration, read from external sources, or template DAG structures dynamically — enabling entire families of pipelines to be defined with minimal code.

Periodic Re-Parsing

The scheduler re-parses DAG files on a configurable interval (default: 30s), automatically detecting new DAGs, modified schedules, and removed files without requiring a restart.

Programmatic Flexibility

DAGs can be generated from config files, database queries, or environment variables — allowing dynamic workflow creation that scales with your data infrastructure needs.

Parse Performance

Complex or slow-parsing DAG files can impact scheduler throughput. Avoid heavy imports at module level and prefer lazy loading to keep parse times minimal.

CHAPTER 3

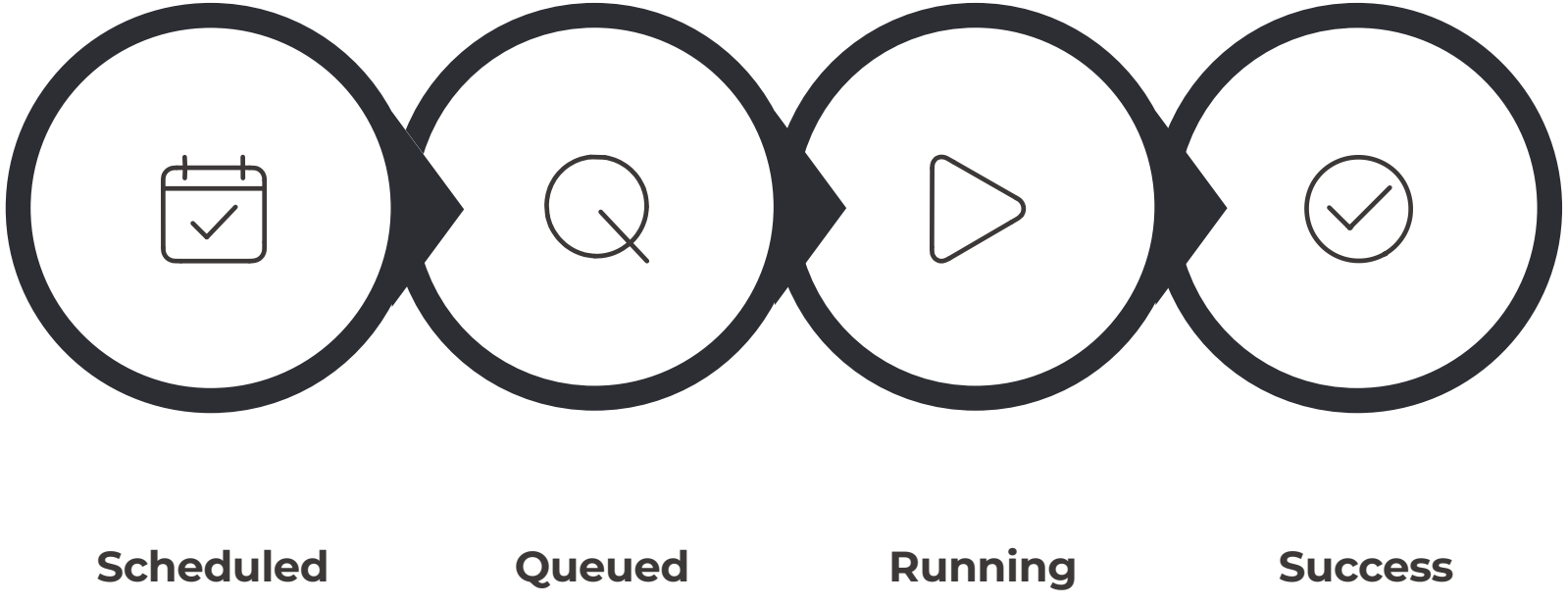
The Journey of a Task

Lifecycle and States

Every task instance travels through a well-defined state machine from creation to completion — or failure. Knowing this lifecycle is essential for debugging and monitoring.

Task Lifecycle: From Conception to Completion

Tasks are the atomic units of work inside a DAG. Their lifecycle — from scheduling eligibility to final outcome — is orchestrated jointly by the **scheduler** (which decides when to run them) and the **executor** (which decides how to run them). Each state transition is persisted in the metadata database in real time.



This state machine ensures Airflow can resume, retry, and track every task instance with full observability — even across distributed worker environments.

Understanding Task States

Queued

Task is eligible and waiting to be picked up by an executor worker.

Running

Task is actively executing on a worker process or pod.

Success

Task completed without errors; downstream tasks may now be scheduled.

Failed

Task encountered an unhandled exception or non-zero exit code.

Upstream Failed

A required upstream task failed, blocking this task from running.

Skipped

Task was bypassed due to branching logic or `skip_on_failure` settings.

Deferred

Task is suspended and waiting for an external event via a Trigger (e.g., a sensor waiting for a file).

Upstream Skipped

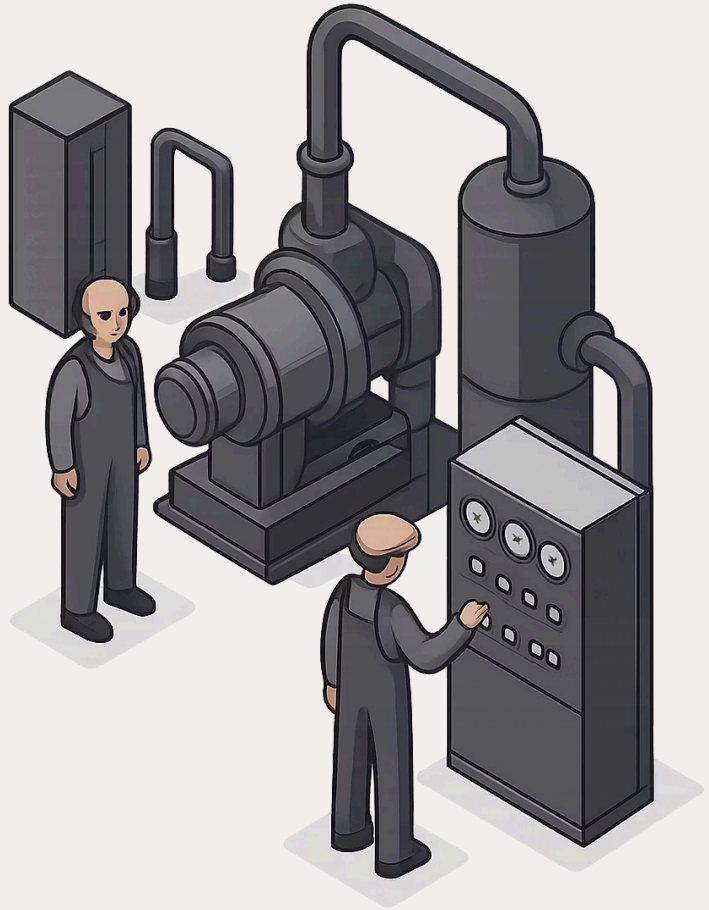
All upstream tasks were skipped, propagating the skip state downstream.

CHAPTER 4

How Tasks Get Done

Executor Models

Executors are the pluggable backends responsible for physically running task instances. Choosing the right executor is one of the most impactful architectural decisions in any Airflow deployment.



The Engine Room: Executor Models

What Is an Executor?

An executor is the component that takes a queued TaskInstance and physically runs it — either locally or on a remote system. Executors are **configured once** per Airflow environment via the `executor` key in the `[core]` section of `airflow.cfg`.

Critically, the executor's **logic runs inside the scheduler process itself** — it doesn't spawn a separate service. Only the worker processes or pods it manages run externally.

01

Scheduler Queues Task

Scheduler determines task is eligible and hands it off to the configured executor.

02

Executor Dispatches Work

Executor routes the task to the appropriate local process, queue, or pod.

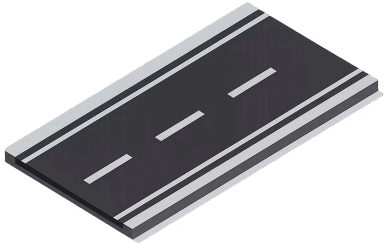
03

Worker Reports Back

Task outcome (success/failure) is written back to the metadata database.

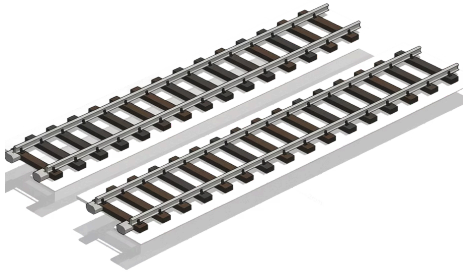
Local Executors

Local executors run tasks on the same machine as the scheduler. They are simple to configure and ideal for development, testing, or small-scale single-machine deployments — but they cannot scale horizontally across multiple nodes.



SequentialExecutor

Executes one task at a time in strict sequence within a single process. Uses **SQLite** as the default database. Zero setup required — perfect for local development and testing, but never suitable for production due to complete lack of parallelism.



LocalExecutor

Spawns a separate **subprocess** for each task, enabling true parallelism on a single machine. Supports configurable `parallelism` limits. Well-suited for small teams and single-server deployments where horizontal scaling is not yet required.

Remote Executors

Remote executors distribute task execution across external infrastructure, enabling horizontal scaling, fault isolation, and production-grade reliability. The right choice depends on your existing infrastructure and operational requirements.

CeleryExecutor

Pushes tasks to a **message broker** (Redis or RabbitMQ). Remote Celery workers pull tasks from the queue and execute them independently. Mature, battle-tested, and highly configurable. Requires broker and result backend infrastructure.

KubernetesExecutor

Launches each task as an isolated **Kubernetes Pod**. Provides perfect resource isolation, dynamic scaling, and native K8s integration. Ideal for cloud-native environments. No persistent workers — pods are created and destroyed per task.

DaskExecutor

Submits tasks to a **Dask distributed cluster** for execution. Particularly well-suited for data science workflows involving heavy numerical computation and existing Dask infrastructure. Less commonly used in production ETL pipelines.

CHAPTER 5

The Brains of the Operation

Metadata Database Schema

Every state change, every DAG run, every task outcome — it all lives in the metadata database. Understanding the schema unlocks deep observability and debugging capability.

Storing the State of Workflows

The Airflow metadata database is the authoritative source of truth for everything happening in your environment. All scheduler decisions, executor updates, and UI queries flow through it. Airflow supports **PostgreSQL** and **MySQL** for production use.

Table	Purpose
`dag`	Stores metadata for all registered DAGs: schedule interval, active status, tags, and ownership information.
`dag_run`	Represents a specific execution instance of a DAG, including run type (scheduled, manual, backfill), logical date, and run state.
`task_instance`	Tracks the current state, start time, end time, duration, and log URL for every individual task execution.
`job`	Records heartbeat activity for scheduler, triggerer, and worker jobs — used for HA leader election and health monitoring.
`sla_miss`	Logs instances where tasks or DAGs failed to complete within their configured SLA time windows.

CHAPTER 6

The Control Panel

Airflow Configuration Structure

Airflow's behavior is deeply configurable — from parallelism limits to authentication backends. Understanding the configuration system lets you tune Airflow precisely for your environment.

Tailoring Airflow to Your Needs

Airflow is configured via the `airflow.cfg` file, which is organized into named sections. Every setting can be overridden using environment variables following the pattern `AIRFLOW__SECTION__KEY` — making it ideal for containerized and secrets-managed deployments.

[core]

General settings: executor type, DAGs folder path, parallelism limits, logging level, and timezone configuration.

[database]

Metadata database connection string (`sql_alchemy_conn`), connection pool size, and query timeout settings.

[scheduler]

Scheduling loop intervals, DAG file processor timeouts, max threads, and HA heartbeat timing.

[webserver]

Web UI host, port, authentication backend, secret key, and session lifetime configuration.

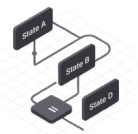
Pro tip: Always use environment variables (`AIRFLOW__CORE__EXECUTOR`) in production to avoid storing sensitive values in plain-text config files and to support 12-factor app deployment patterns.

Mastering Airflow Internals

A deep understanding of Airflow's core components transforms you from a pipeline author into a platform engineer — capable of optimizing, troubleshooting, and scaling Airflow with confidence.

Scheduler & DAG Parsing

Know how the scheduler loop works and how DAGs are parsed to write efficient, production-safe workflows that don't strain your infrastructure.



Task Lifecycle & States

Understanding every task state empowers you to build robust retry logic, diagnose failures faster, and design resilient pipelines from the ground up.

Executors & Configuration

Choosing the right executor and tuning configuration for your environment is the difference between a fragile prototype and a scalable, enterprise-grade Airflow deployment.